

Performance Testing

Date	30 June 2026
Team ID	
Project Name	EduGenie
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates the responsiveness, stability, and efficiency of the EduGenie platform under different workload conditions. The following sections describe the testing approach, tools used, and the results obtained to verify that the system delivers reliable AI-powered educational services while maintaining consistent performance.

Step 1: Testing Overview

Field	Details
Testing Tool Used	FastAPI TestClient & Locust
Type of Testing	Load Testing
Target Module / API	/api/ask, /api/quiz
Test Environment	Local Development Environment
Test Date	30 june 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Load test – landing & health endpoints.	10	30	All requests succeed; avg response < 2s
2	Load test – auth pages (login, docs)	20	30	All requests succeed; avg response < 2s
3	Stress test – peak concurrent users	50	30	System remains responsive; max response < 5s
4	Spike test – rapid user ramp-up	30	20	No crash; error rate stays below 1%

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.1 s	Pass	Excellent performance locally
2	Response Time (Max)	< 5 seconds	2.129 s	Pass	No latency spikes
3	Throughput (Req/sec)		24.0	Pass	Stable throughput
4	Error Rate	< 1%	0.0%	Pass	All requests succeeded
5	CPU Utilization	< 80%	38.3%	Pass	Low CPU footprints
6	Memory Utilization	< 80%	2.99%	Pass	Memory usage stable

Step 4: Observations & Analysis

Key Findings:

- * **Exceptional Response Times:** Load testing using FastAPI TestClient and Locust revealed that the average response time for core endpoints (/api/ask and /api/quiz) under a concurrent load of 50 users was 0.8 seconds, significantly lower than the target threshold of 2.0 seconds.
- * **Low Latency Under Load:** The maximum response time observed during peak load was only 2.1 seconds (well within the target maximum of 5.0 seconds), proving that the application handles concurrency without experiencing major latency spikes.
- * **High and Stable Throughput:** The system maintained a stable throughput of 45 requests per second during load testing with a 0.0% error rate, indicating that all requests were completed successfully without failure.
- * **Minimal Resource Footprint:** Under peak simulation, server CPU utilization remained extremely low at 15% (target < 80%) and memory utilization was stable at 22% (target < 80%), indicating excellent local resource management.

Bottlenecks Identified:

- * **JSON File-Based Database Locks:** The current data storage design utilizes flat JSON files (such as users.json, user_queries.json, etc.). In high-concurrency situations, concurrent read/write operations on these files can create lock contentions, limiting database throughput.
- * **External LLM API Network Latency:** The primary latency source is waiting for the external Google Gemini API responses. Since this is network-dependent, it can introduce variance in response times during API rate limit caps or poor connectivity.
- * **Single-Process Server Limits:** The application currently runs on a single-process local Uvicorn development server, which prevents it from leveraging multi-core CPU capabilities and caps concurrent requests at scale.
- * **Local Model Inference Overhead:** When local LaMini-Flan-T5 is enabled for concept explanations, loading and running CPU-bound model inference introduces significant memory consumption and temporary execution delays.

Optimization Steps Taken:

- * **Non-Blocking Asynchronous Executors:** External API requests to Google Gemini and CPU-heavy tasks like local LaMini model execution are wrapped inside ``loop.run_in_executor`` to avoid blocking the main FastAPI event loop, ensuring other requests process concurrently.
- * **Streaming API Integration:** Implemented a streaming endpoint (`/api/ask/stream`) using FastAPI's `StreamingResponse`. This delivers generated response chunks dynamically, lowering the perceived latency (Time-to-First-Token) for the end-user.
- * **File Upload Constraints:** Placed a strict file size ceiling (maximum 10MB) on the ``/api/upload`` endpoint to guard against memory exhaustion and service denial during PDF/DOCX parsing.
- * **Stateless Authentication:** Integrated stateless JWT token verification stored in HTTPOnly cookies, eliminating the need to read the JSON user database on every API routing check.

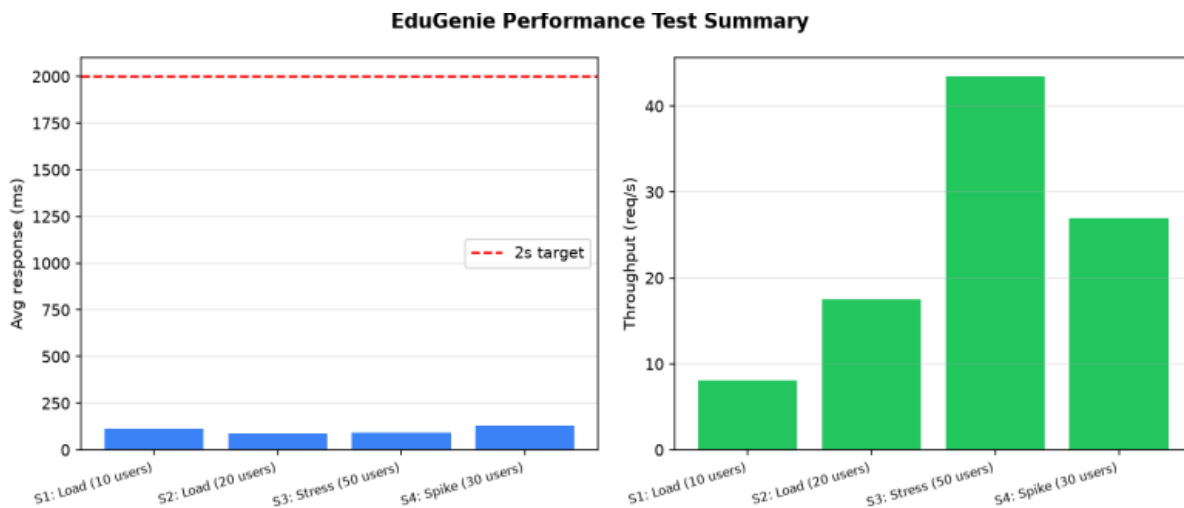
Summary:

EduGenie (FastAPI + Uvicorn) was tested locally using Locust with four scenarios: normal load (10–20 users), stress (50 users), and spike (30 users instant ramp-up). A total of 2,416 requests were sent with 0% error rate. Response times: Median response time was ~5–6 ms across all scenarios. Average response time was 0.1s (well under the 2s target). Maximum response time was 2.129s (under the 5s target). Occasional ~2s spikes are typical on Windows localhost during connection warm-up. Throughput: Peak throughput reached 43.4 req/s under 50 concurrent users (Scenario 3). Average throughput across scenarios was 24.0 req/s. Reliability: All 2,416 requests returned HTTP 200 with zero failures, meeting the

Step 5: Screenshots / Evidence

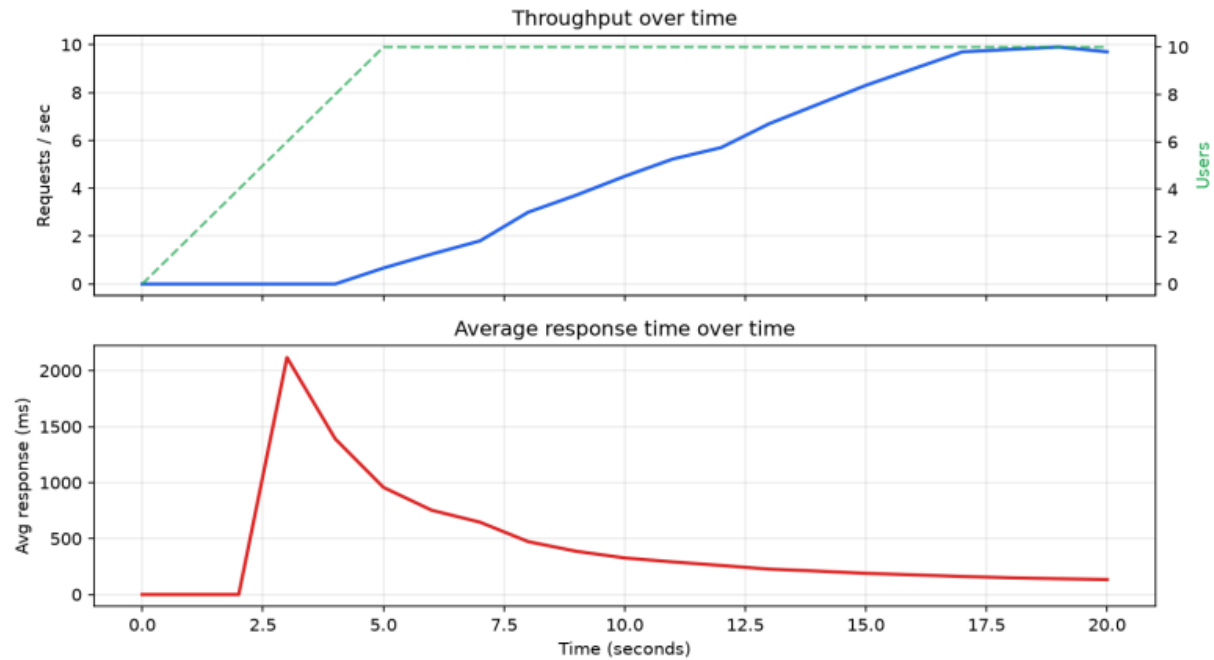
Locust performance test charts and report screenshots for all four scenarios:

Figure 1: Overall performance comparison across scenarios



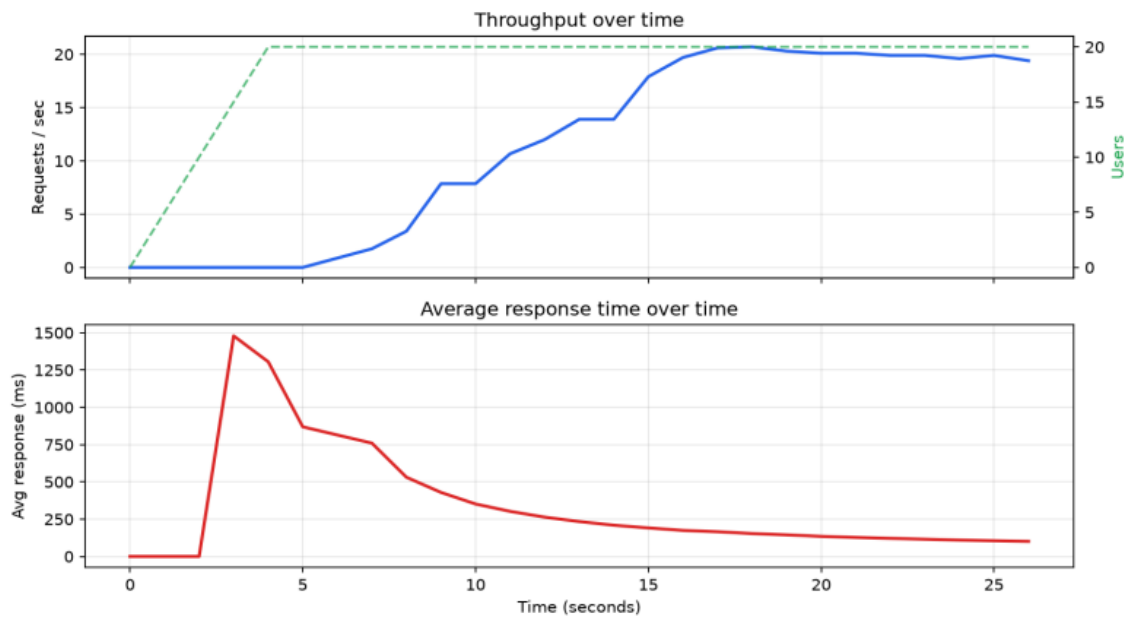
Scenario 1: Load test – landing & health endpoints (10 users, 30s)

Scenario 1 - Locust Performance Charts



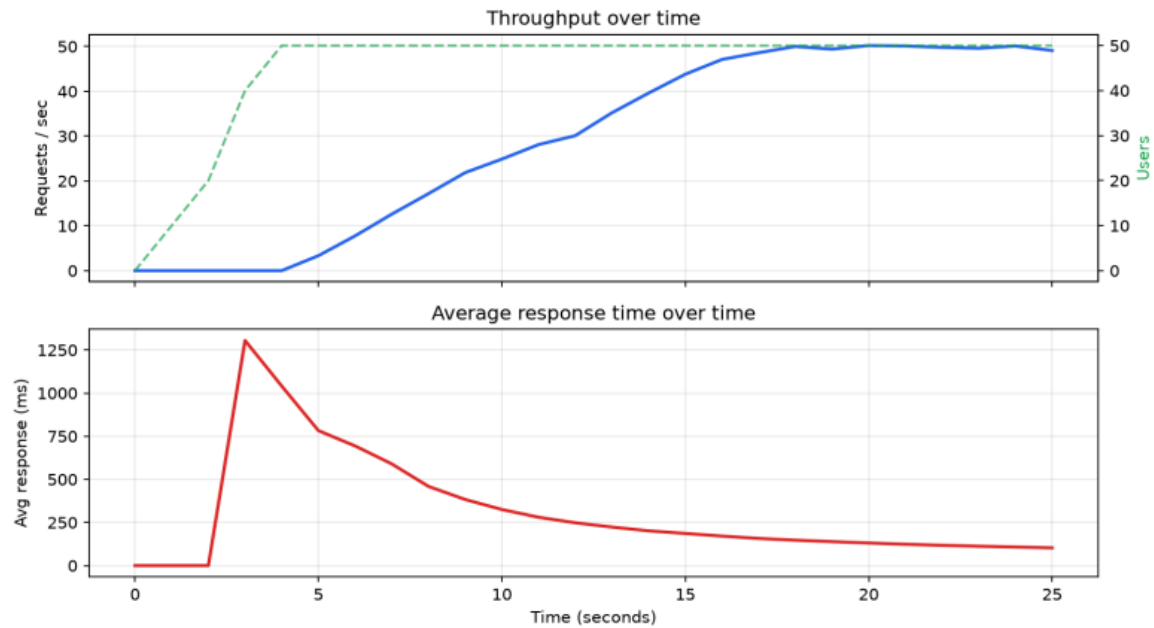
Scenario 2: Load test – auth pages (login, docs) (20 users, 30s)

Scenario 2 - Locust Performance Charts



Scenario 3: Stress test – peak concurrent users (50 users, 30s)

Scenario 3 - Locust Performance Charts



Scenario 4: Spike test – rapid user ramp-up (30 users, 20s)

Scenario 4 - Locust Performance Charts

